



Pemrograman Piranti Bergerak

I Made Pasek Pradnyana Wijaya, S.Kom., M.Kom



07

- Read Delete

08

- Sensor Piranti Bergerak

09-10

- Sensor Piranti Bergerak

11-13

- **Web Services**

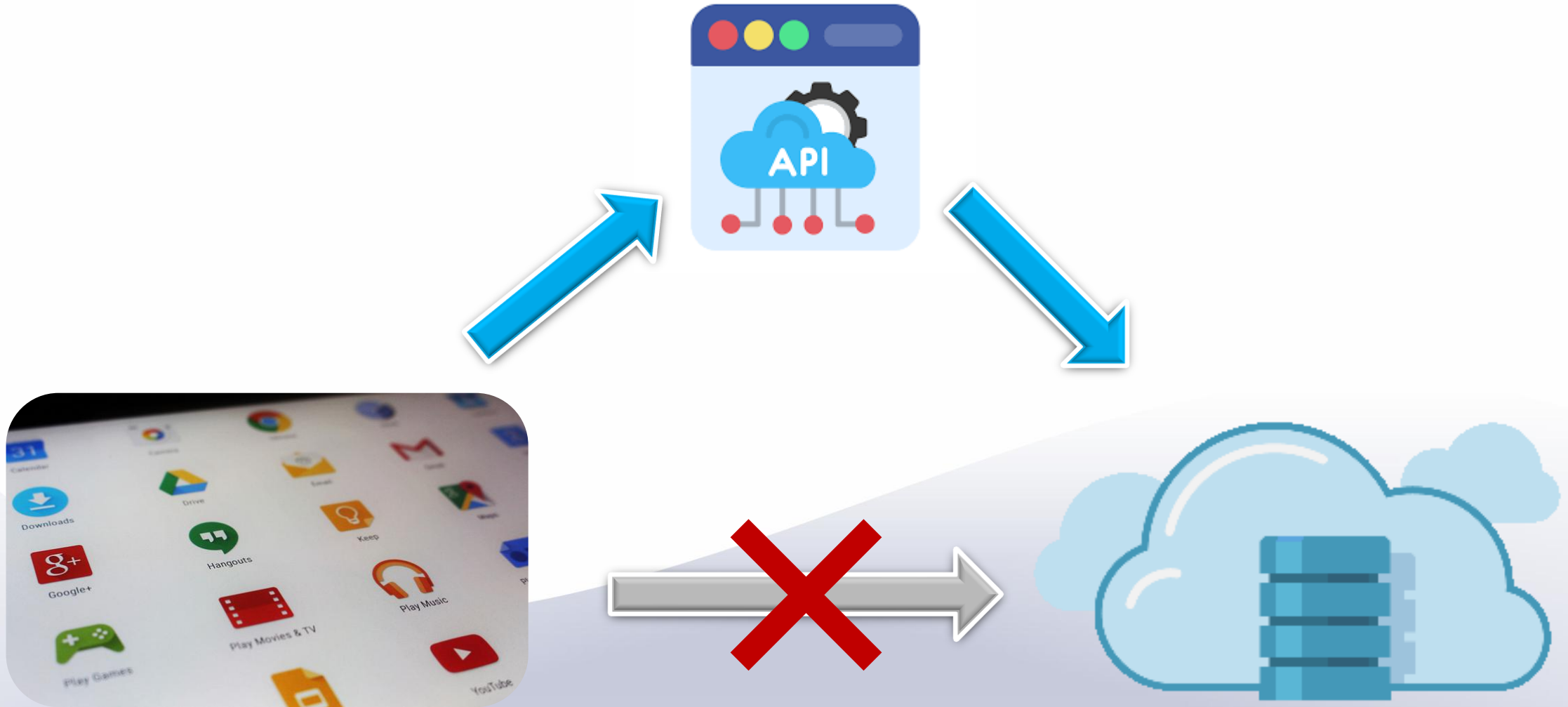


Cloud Database





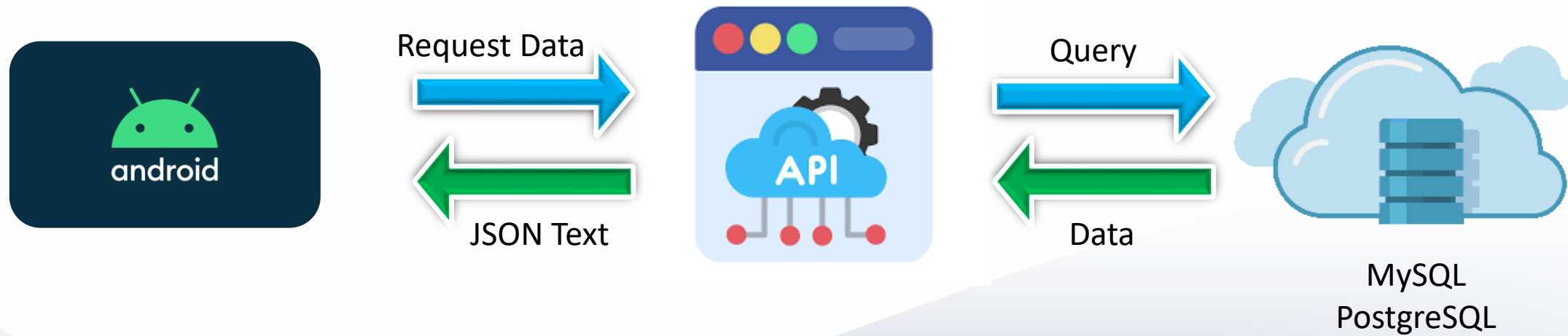
Konsep Cloud Database





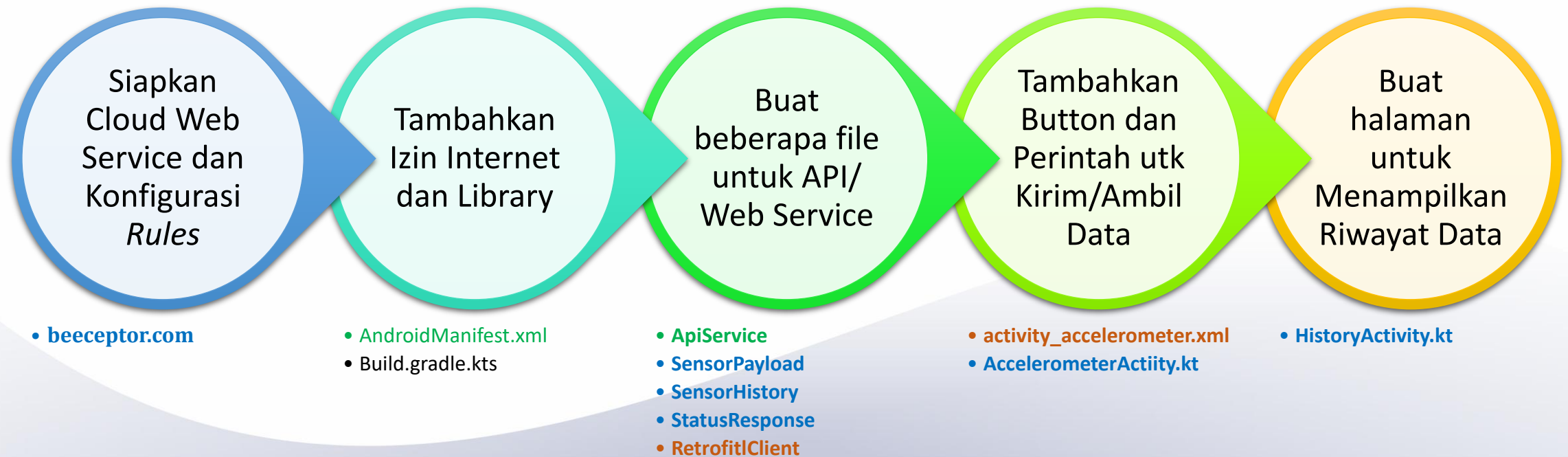
Konsep Web Service (API)

Web Service adalah suatu “jembatan” yang menghubungkan suatu aplikasi ke database





Tahapan Implementasi Sensor Accelerometer





beeceptor.com

Akses beeceptor.com kemudian isikan field nama Mock Server dengan nama unik

The screenshot shows the Beeceptor website interface. The main heading is "AI-Powered Mock Servers in Seconds". Below it, a subheading reads "Build REST, GraphQL, SOAP, and gRPC servers in a few seconds." A button labeled "Watch to get started" is visible. A modal window titled "Unblock your day" is open, featuring a text input field containing "latihansensor" and ".free.beeceptor.com". The input field is circled in red. Below the input field is a purple button labeled "Create Mock Server". At the bottom of the modal, it says "Open recent: #stikom-bb231, #stikom-senin-apps". The website's navigation bar includes links for "Skip the Waiting", "Mock Servers", "Tools", "Features", "Docs", "Pricing", and "Made Pa...".



beeceptor.com

Setelah berhasil dibuat, akan ada Alamat URL Virtual Mock Server.
Lakukan konfigurasi Mock Rules untuk kirim dan ambil data

The screenshot shows the Beeceptor web interface in a browser. The address bar shows `app.beeceptor.com/console/latihansensor`. The main heading is `#latihansensor`. Below it, the URL `https://latihansensor.free.beeceptor.com → {nowhere}` is displayed. A red arrow points to this URL. To the right, there are tabs for `Mock Rules (2)` and `Routing`, with a red arrow pointing to the `Mock Rules (2)` tab. The central message says "Looks Awesome!" and "Your server's ready! Copy the URL below and drop it into your app as the base URL. Create mock rules to design your API paths and responses just the way you need!". Below this, the URL `https://latihansensor.free.beeceptor.com` is shown with a copy icon. A status indicator shows a green circle and the text "Ready and waiting! Fire off your HTTPs or API requests." At the bottom, there are three cards: "Watch: How Beeceptor Works", "Send a Sample GET Request", and "Design Your API". On the right side, a sidebar titled "Get started with Beeceptor" contains a progress bar and several options: "Create a Mock API" (checked), "Test Your API", "Inspect Requests", and "Generate Dummy Data".



beeceptor.com

Pertama, buat Rule baru **/ kirim-data** dengan Method **POST**. Isikan data sesuai gambar dibawah

Mock Rules

When following condition is matched (for request)

Method

POST

Request condition

Request path exactly matches

Match value/expression

/kirim-data

+ Add condition

Do the following (for response)

☒ Single response ☐ Weighted response

Response delayed by

0

sec

Return HTTP status as

200

Response headers

Content-Type

application/json

+ Add Header

Response body

Insert Dynamic Value

{

"status": "Sukses",

"pesan": "Data tersimpan!"

}

{ }

Send a file/blob

Cancel

✓ Save

imppw



beeceptor.com

Kedua, buat Rule baru **/ambil-data** dengan Method **GET**. Isikan data sesuai gambar dibawah

The screenshot shows the 'Mock Rules' configuration window in Beeceptor. It is divided into two main sections: 'When following condition is matched (for request)' and 'Do the following (for response)'.

When following condition is matched (for request):

- Method:** GET (selected from a dropdown)
- Request condition:** Request path exactly matches (selected from a dropdown)
- Match value/expression:** /ambil-data (entered in a text field)
- + Add condition** (link)

Do the following (for response):

- Single response** (selected radio button) / **Weighted response** (radio button)
- Response delayed by:** 0 (text field) **sec** (unit dropdown)
- Return HTTP status as:** 200 (text field)
- Response headers:**
 - Content-Type:** application/json (text field)
 - + Add Header** (link)
- Response body:**
 - Insert Dynamic Value** (link)
 - Send a file/blob** (checkbox, unchecked)
 - Response body content:**

```
[
  {
    "id": 1,
    "timestamp": "12.00",
    "kemiringan_x": 0.52,
    "status_setir": "Lurus"
  }
]
```

At the bottom, there are **Cancel** and **Save** buttons.



AndroidManifest.xml

Buka file AndroidManifest.xml dan tambahkan izin untuk mengakses internet

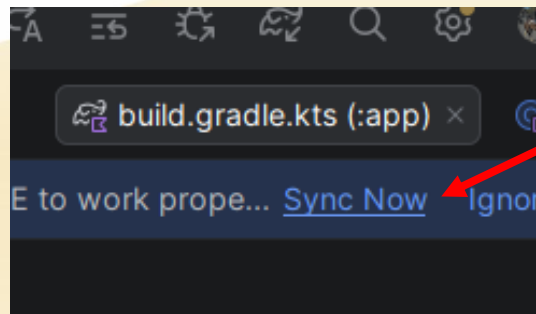
```
</> activity_accelerometer.xml AccelerometerActivity.kt AndroidManifest.xml x MainActivity.kt
1  <?xml version="1.0" encoding="utf-8"?>
2  <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:tools="http://schemas.android.com/tools">
4  ⚡ <uses-permission android:name="android.permission.INTERNET" />
5
```



build.gradle.kts (Module: app)

Buka file **build.gradle.kts** dan tambahkan dua perintah dibawah. Kemudian klik **Sync Now**

```
38 dependencies {
39     implementation(libs.androidx.core.ktx)
40     implementation(libs.androidx.appcompat)
41     implementation(libs.material)
42     implementation(libs.androidx.activity)
43     implementation(libs.androidx.constraintlayout)
44     testImplementation(libs.junit)
45     androidTestImplementation(libs.androidx.junit)
46     androidTestImplementation(libs.androidx.espresso.core)
47
48     implementation("com.squareup.retrofit2:retrofit:2.9.0")
49     implementation("com.squareup.retrofit2:converter-gson:2.9.0")
50 }
```





build.gradle.kts (Module: app) - Retrofit

Apa Itu Retrofit

- Library yang sifatnya open-source yang digunakan untuk platform Android/Java/Kotlin yang digunakan sebagai penghubung antara aplikasi Android dengan Server

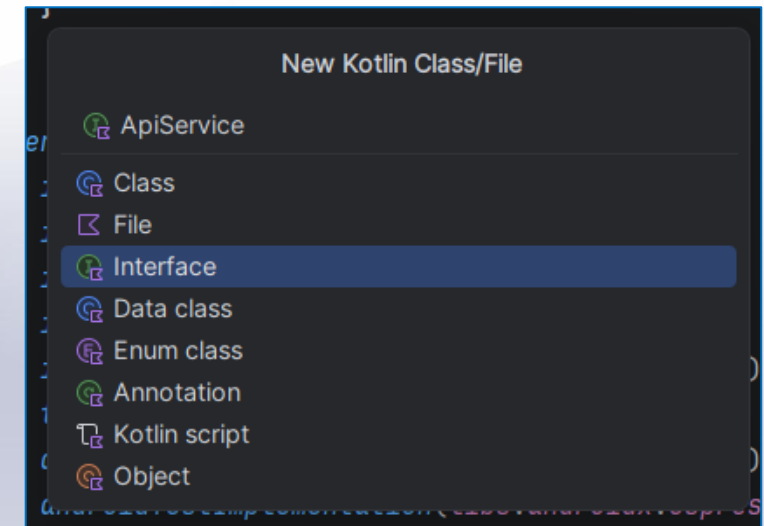
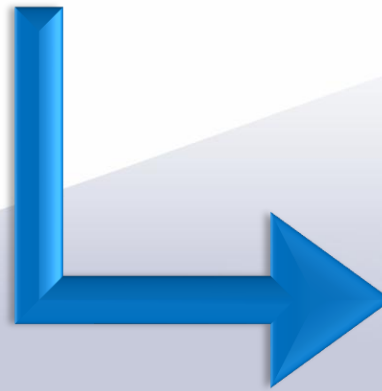
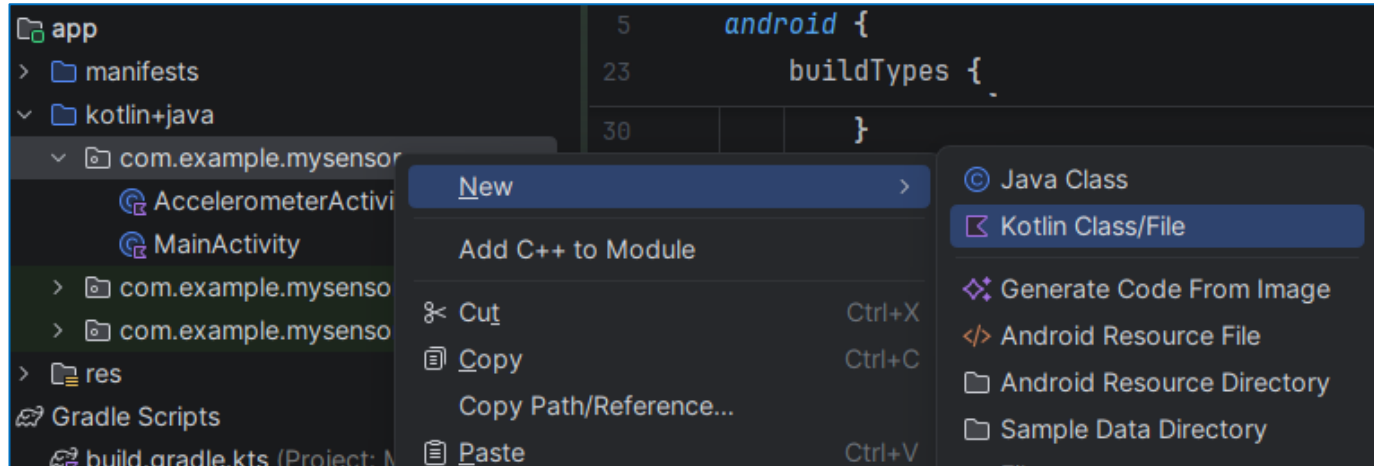
Kegunaan Retrofit

- Sebagai API/Web Service yang akan mengirimkan data maupun mengambil data dari server



ApiService.kt

Buat satu file **Kotlin Class/File** baru dengan tipe **Interface**, beri nama **ApiService**





ApiService.kt

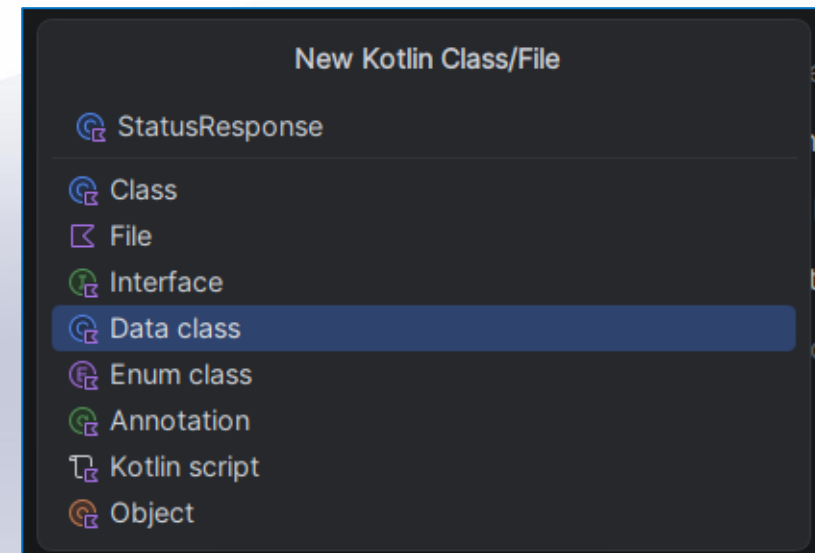
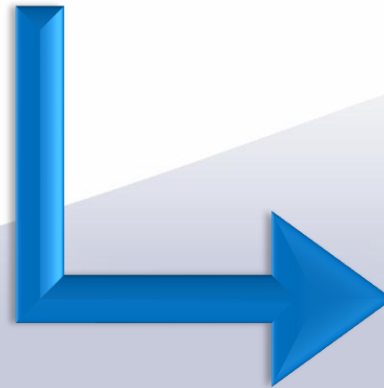
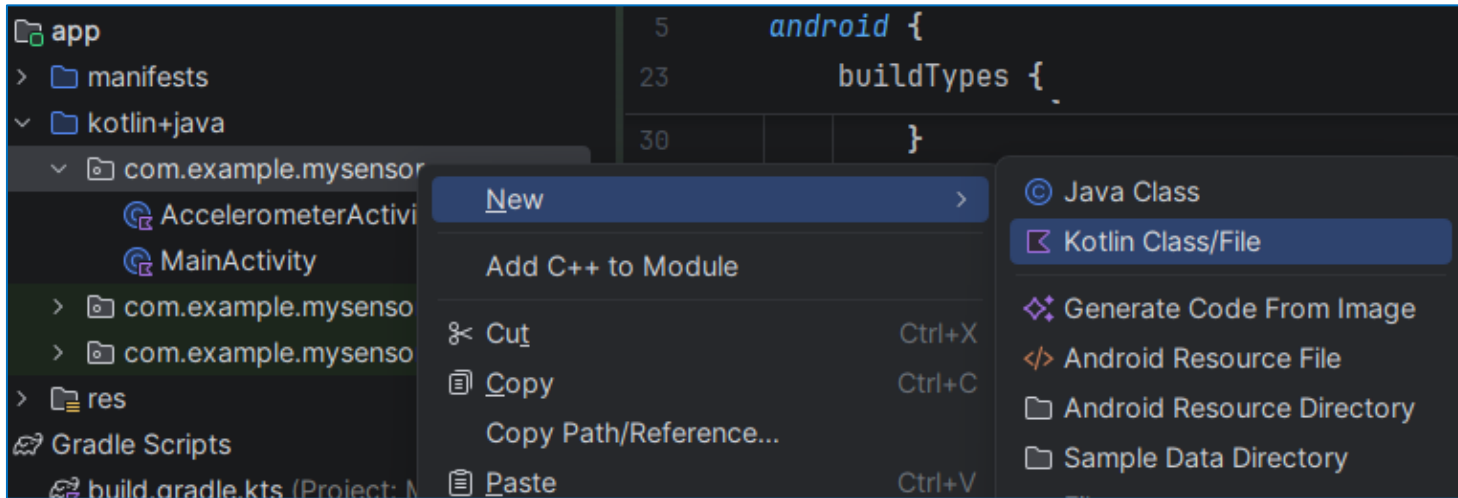
ApiService berisi perintah untuk Mengirim Data (Method **@POST**) yg diarahkan ke Rules **/ kirim-data**, dan perintah untuk Mengambil Data (Method **@GET**) yg diarahkan ke Rules **/ambil-data**

```
ApiService.kt x
1  package com.example.mysensor
2
3  import retrofit2.Call
4  import retrofit2.http.Body
5  import retrofit2.http.GET
6  import retrofit2.http.POST
7
8  interface ApiService {
9      @POST("kirim-data")
10     fun kirimDataSensor(@Body payload: SensorPayload): Call<StatusResponse>
11
12     @GET("ambil-data")
13     fun ambilDataSensor(): Call<List<SensorHistory>>
14 }
```



StatusResponse.kt

Buat satu file **Kotlin Class/File** baru, dengan tipe **Data class**, beri nama **StatusResponse**





StatusResponse.kt

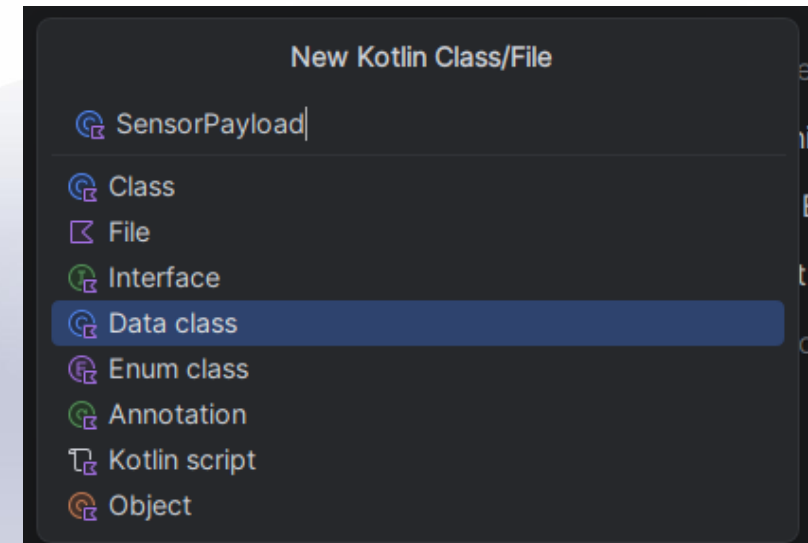
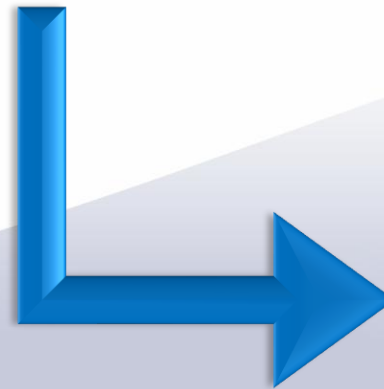
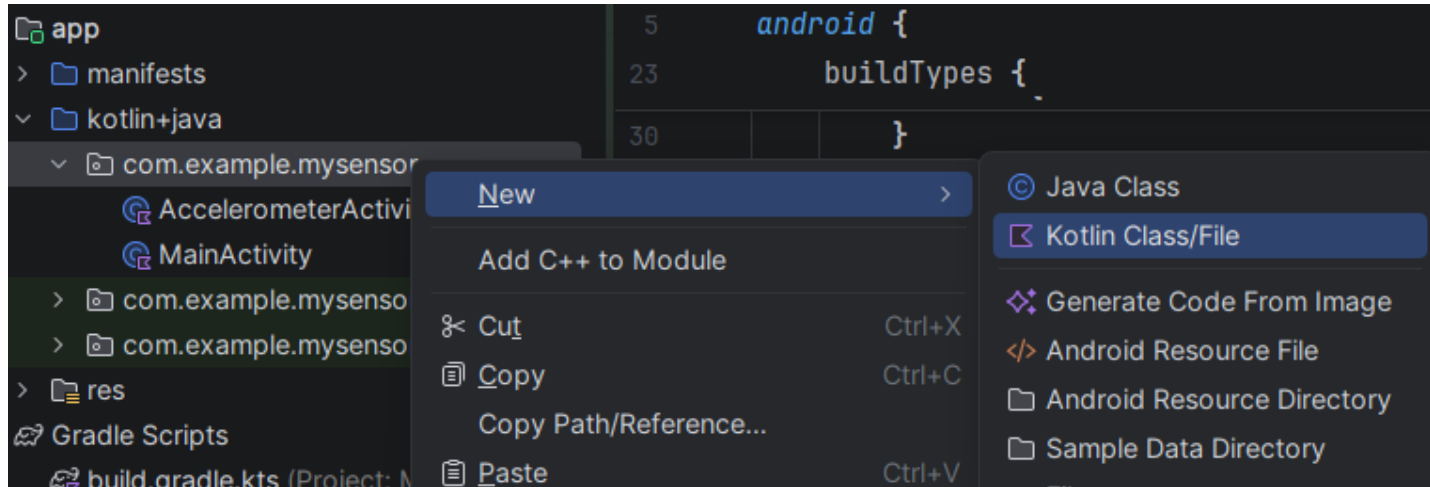
File ini berisi sebagai “tanda terima” dari server setelah aplikasi berhasil mengirim data

```
StatusResponse.kt x
1 package com.example.mysensor
2
3 data class StatusResponse(
4     val status: String,
5     val message: String,
6 )
```



SensorPayload.kt

Buat satu file **Kotlin Class/File** baru, dengan tipe **Data Class**, beri nama **SensorPayload**





SensorPayload.kt

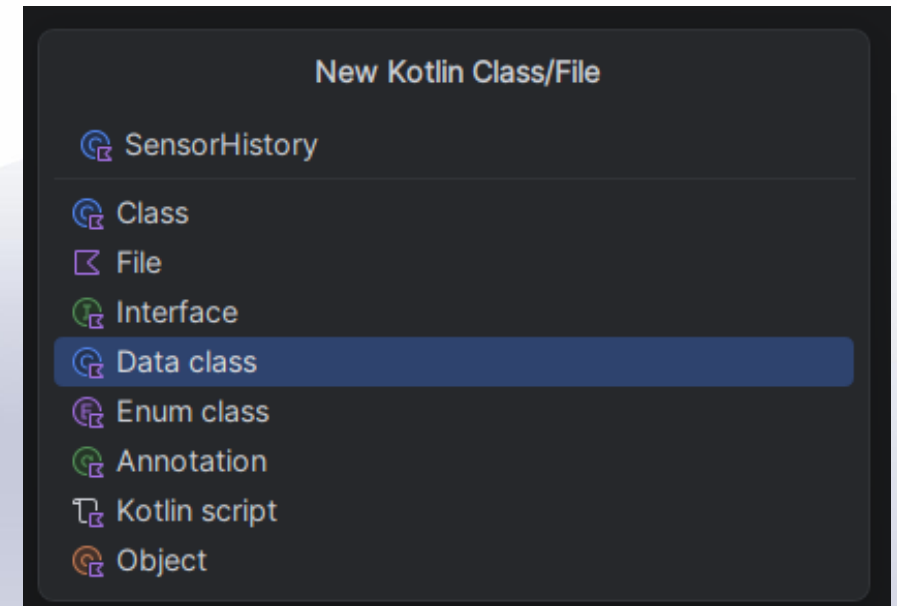
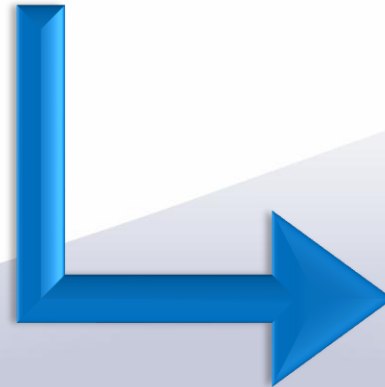
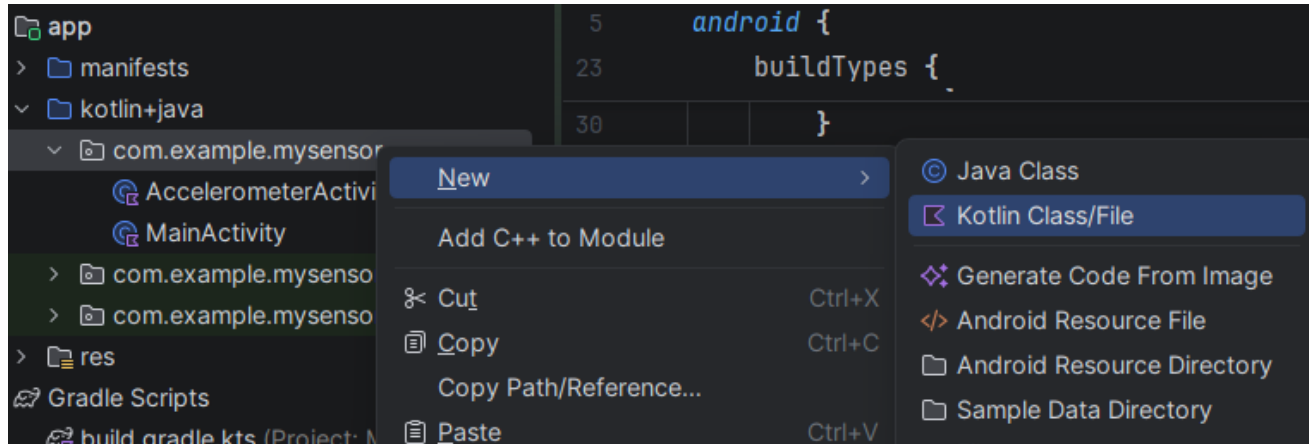
SensorPayload digunakan untuk “membungkus” data yang ingin dikirim ke server (**POST**)

```
SensorPayload.kt x
1 package com.example.mysensor
2
3 data class SensorPayload(
4     val kemiringan_x: Float,
5     val status_setir: String
6 )
7
```



SensorHistory.kt

Buat satu file **Kotlin Class/File** baru, dengan tipe **Data Class**, beri nama **SensorHistory**





SensorHistory.kt

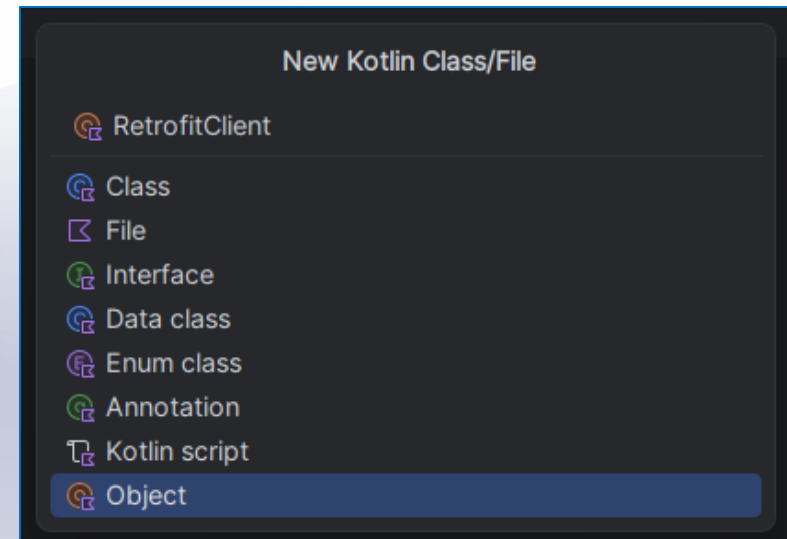
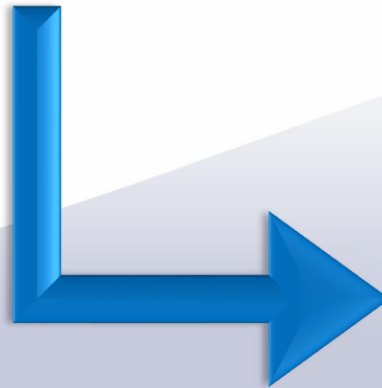
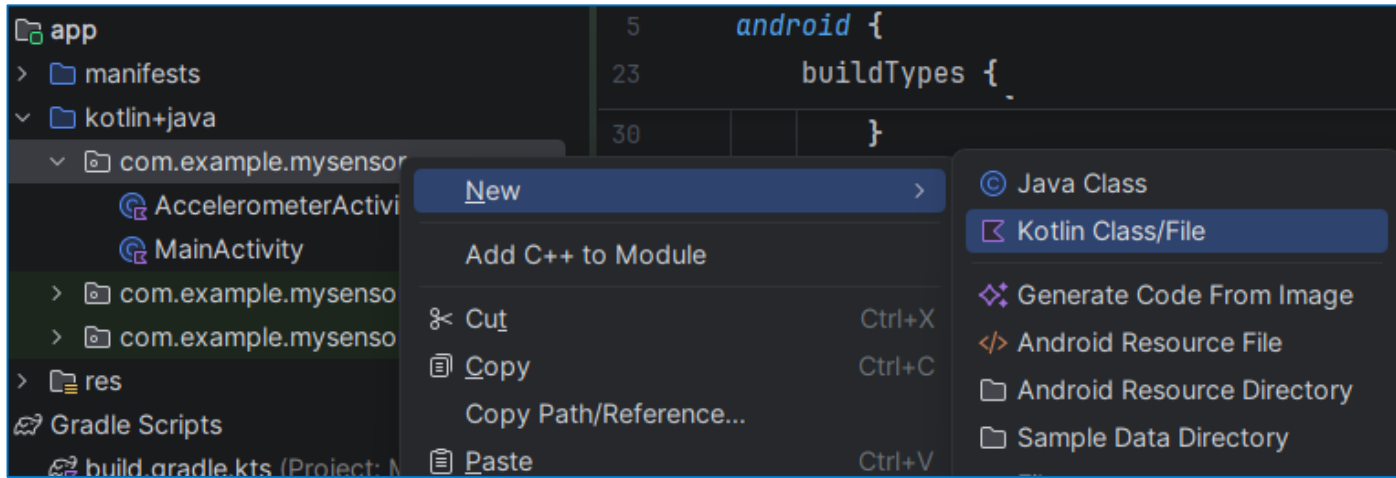
SensorHistory digunakan untuk menampung data yang diambil dari server (**GET**)

```
SensorHistory.kt x
1 package com.example.mysensor
2
3 data class SensorHistory(
4     val id: Int,
5     val kemiringan_x: Float,
6     val status_setir: String,
7     val timestamp: String
8 )
9
```



RetrofitClient.kt

Buat satu file **Kotlin Class/File** baru, dengan tipe **Object**, beri nama **RetrofitClient**





RetrofitClient.kt

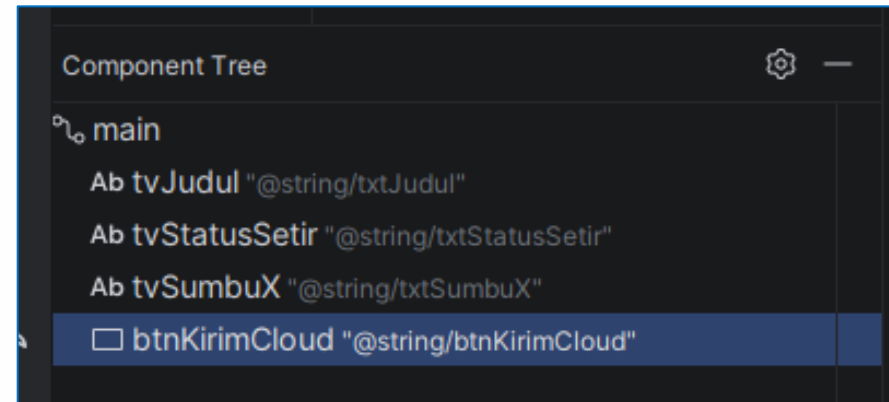
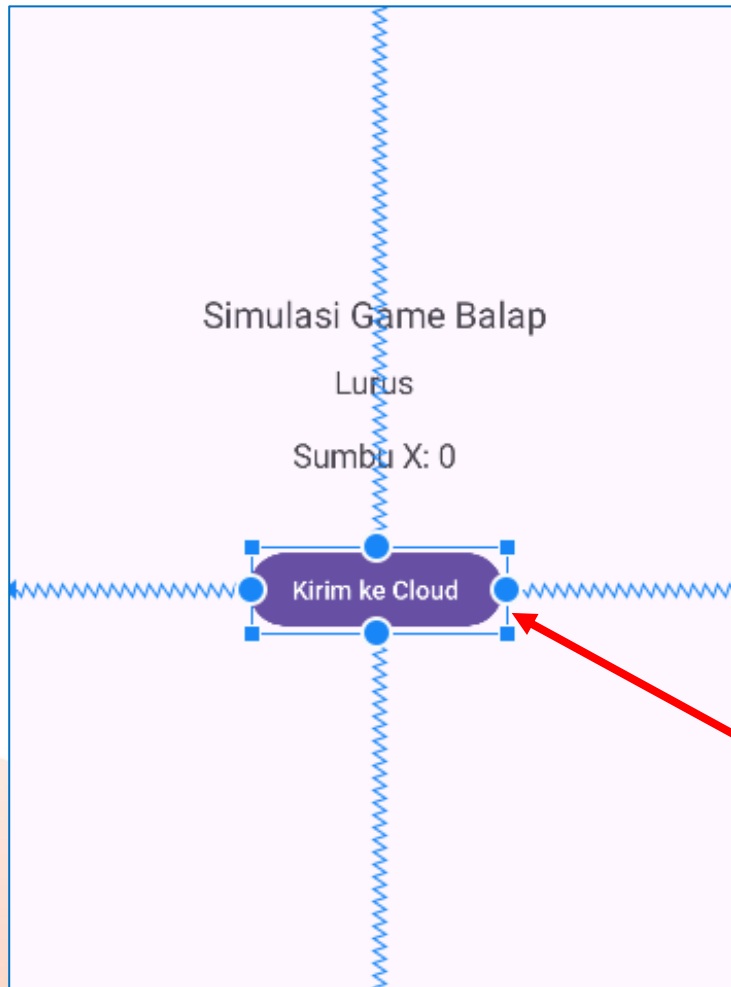
File **RetrofitClient** ini fungsinya adalah konfigurasi utama untuk mengatur koneksi aplikasi dengan server yang ada. Masukkan alamat Mock Server yang sebelumnya sudah dibuat

```
RetrofitClient.kt x
1  package com.example.mysensor
2
3  import retrofit2.Retrofit
4  import retrofit2.converter.gson.GsonConverterFactory
5
6  object RetrofitClient {
7      private const val BASE_URL = "https://latihansensor.free.beeceptor.com/"
8
9      val instance: ApiService by lazy {
10          val retrofit = Retrofit.Builder()
11              .baseUrl(BASE_URL)
12              .addConverterFactory(GsonConverterFactory.create())
13              .build()
14
15          retrofit.create(ApiService::class.java)
16      }
17  }
```



activity_accelerometer.xml

Pada halaman activity_accelerometer, tambahkan **Button** untuk pindah ke halaman History





AccelerometerActivity.kt

Pada file **AccelerometerActivity**, import beberapa perintah dan tambahkan variable utk **Button**, dan juga dua variable untuk data yang akan dikirim (**nilaiX** dan **statusSetir**)

```
15
16     import android.widget.Button
17     import android.widget.Toast
18     import retrofit2.Call
19     import retrofit2.Callback
20     import retrofit2.Response
21
```

```
22     class AccelerometerActivity : AppCompatActivity(), SensorEventListener {
26         private lateinit var tvStatus: TextView
27         private lateinit var tvSumbuX: TextView
28         private lateinit var btnKirimCloud: Button
29
30         private var nilaiX: Float = 0f
31         private var statusSetir: String = "Lurus"
32
```



AccelerometerActivity.kt

Dibawah **function onAccuracyChanged()**, tambahkan **function kirimDataKeCloud()**

```
70  override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}
71
72  private fun kirimDataKeCloud(nilaiX: Float, statusSetir: String) {
73      val payload = SensorPayload(nilaiX, statusSetir)
74
75      RetrofitClient.instance.kirimDataSensor(payload).enqueue(object : Callback<StatusResponse> {
76          override fun onResponse(call: Call<StatusResponse>, response: Response<StatusResponse>) {
77              if (response.isSuccessful) {
78                  Toast.makeText(
79                      this@AccelerometerActivity,
80                      "Data berhasil dikirim ke cloud",
81                      Toast.LENGTH_SHORT
82                  ).show()
83              }
84          }
85      })
```



AccelerometerActivity.kt

Berikut ini adalah lanjutan isian dari **function kirimDataKeCloud()** utk **function onFailure()** yg letaknya dibawah **function onResponse()**

```
72     private fun kirimDataKeCloud(nilaiX: Float, statusSetir: String) {
75         RetrofitClient.instance.kirimDataSensor(payload).enqueue(object : Callback<StatusResponse> {
76             override fun onResponse(call: Call<StatusResponse>, response: Response<StatusResponse>) {
83                 }
84             }
85
86             override fun onFailure(call: Call<StatusResponse?>?, t: Throwable?) {
87                 Toast.makeText(
88                     this@AccelerometerActivity,
89                     "Koneksi error: ${t?.message}",
90                     Toast.LENGTH_SHORT
91                 ).show()
92             }
93         })
94     }
95 }
```



AccelerometerActivity.kt

Selanjutnya cari **function onCreate()** dan didalamnya tambahkan perintah trigger **btnKirimCloud** untuk mengirim data dengan memanggil **function kirimDataKeCloud()**

```
33      override fun onCreate(savedInstanceState: Bundle?) {
44          tvStatus = findViewById(R.id.tvStatusSetir)
45          sensorManager = getSystemService(Context.SENSOR_SERVICE) as SensorManager
46          accelerometerSensor = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)
47
48          btnKirimCloud = findViewById(R.id.btnKirimCloud)
49          btnKirimCloud.setOnClickListener {
50              kirimDataKeCloud(nilaiX, statusSetir)
51          }
52
53      }
54      override fun onSensorChanged(event: SensorEvent?) {
```



AccelerometerActivity.kt

Didalam **function onSensorChanged()**, sesuaikan bagian ini sehingga isi perintahnya terutama pada nama variable seperti pada gambar

```
54  override fun onSensorChanged(event: SensorEvent?) {  
55      if(event?.sensor?.type == Sensor.TYPE_ACCELEROMETER) {  
56          nilaiX = event.values[0]  
57          tvSumbuX.text = "Sumbu X: ${String.format("%.2f", nilaiX)}"  
58      }  
59      if(nilaiX < -2) {  
60          statusSetir = "Belok Kanan"  
61      } else if(nilaiX > 2) {  
62          statusSetir = "Belok Kiri"  
63      } else {  
64          statusSetir = "Lurus"  
65      }  
66      tvStatus.text = statusSetir  
67  }  
68  }
```



Simulasi Kirim Data

Run aplikasi dan monitoring **beeceptor** apakah data sudah berhasil terkirim atau belum

Simulasi Game Balap

Belok Kiri

Sumbu X: 8.70

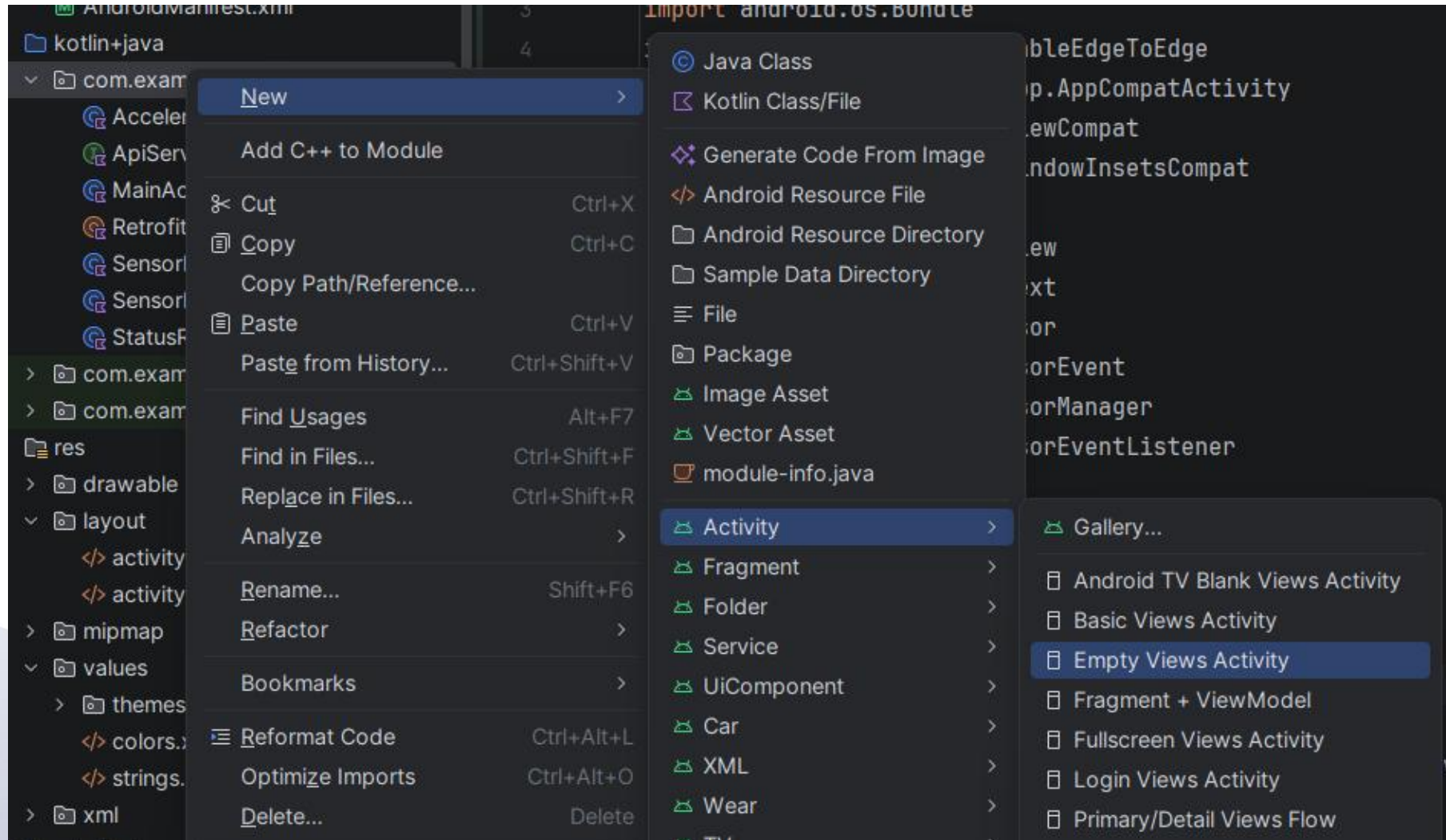
Kirim ke Cloud

POST / kirim-data	200 0.0s a few seconds ago	☒
POST / kirim-data	200 0.0s a few seconds ago	☒
Request Body: <pre>{ "kemiringan_x": 8.705292, "status_setir": "Belok Kiri" }</pre>	Response Body: <pre>{ "status": "Sukses", "pesan": "Data tersimpan!" }</pre>	☒
POST / kirim-data	200 0.0s 7 minutes ago	☒
POST / kirim-data	200 0.0s 7 minutes ago	☒



HistoryActivity.kt

Buat **Activity** baru dengan tipe **Empty Views Activity**





HistoryActivity.kt

Berikan nama *activity* baru tersebut misalkan **HistoryActivity**, yg digunakan untuk menampilkan riwayat data dari server

New Android Activity

Empty Views Activity
Creates a new empty activity

Activity Name
HistoryActivity

☒ Generate a Layout File

Layout Name
activity_history

☐ Launcher Activity

Package name
com.example.mysensor

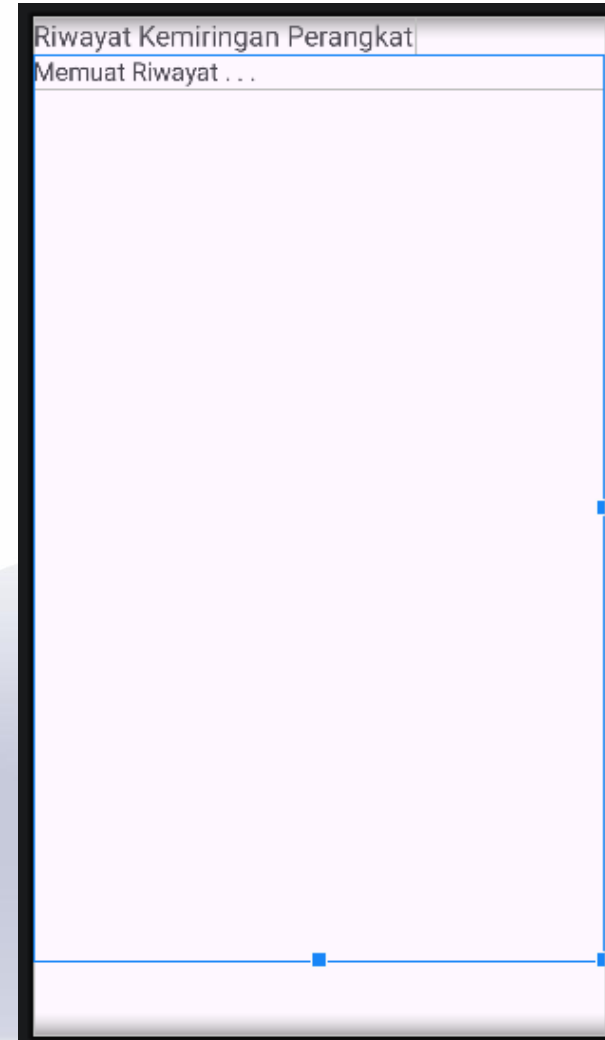
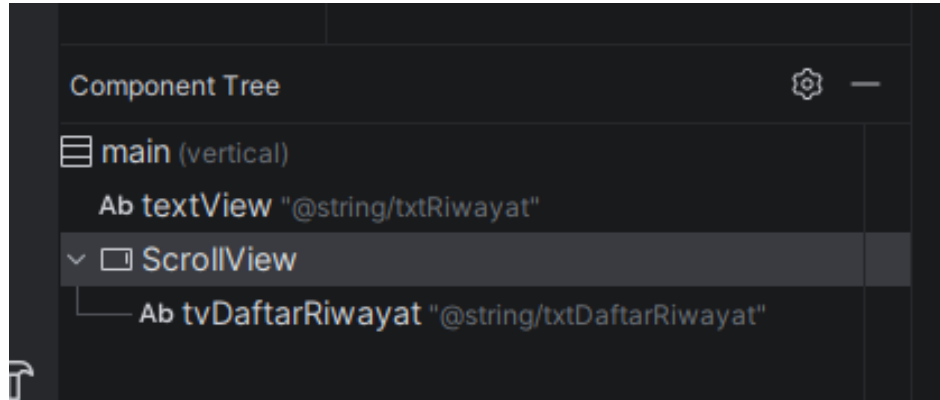
Source Language
Kotlin

Cancel Finish



activity_history.xml

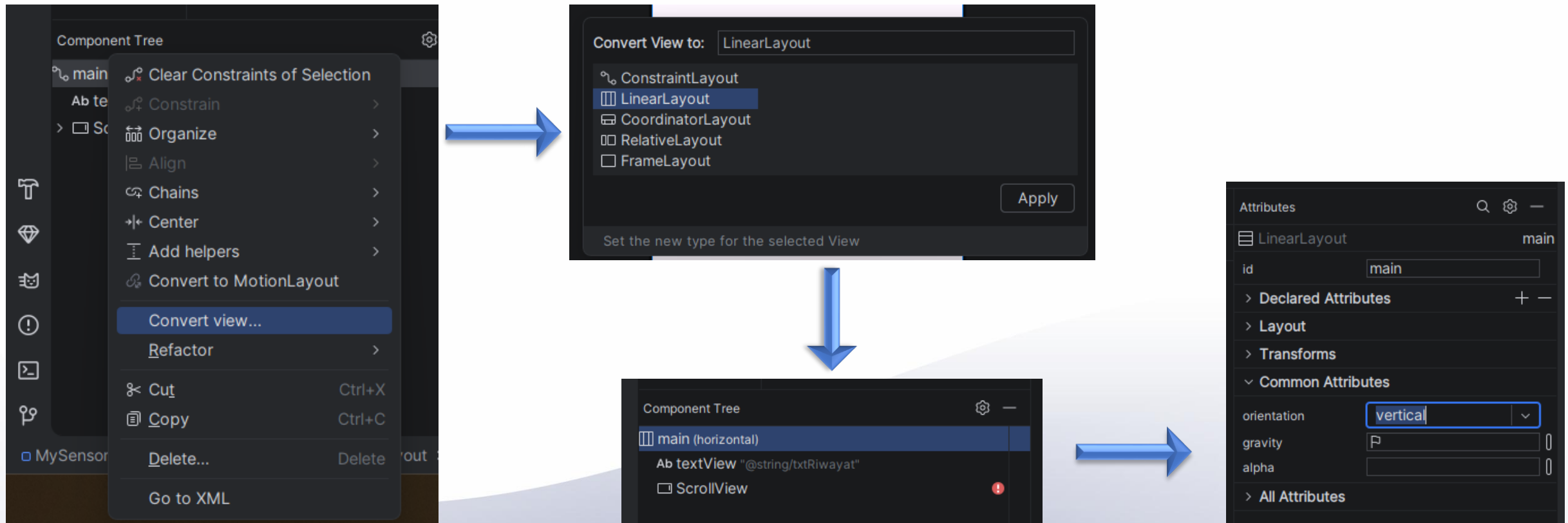
Tambahkan 3 elemen pada halaman **activity_history** untuk menampilkan data





activity_history.xml

Ubah tipe view dari **Main** menjadi **LinearLayout**





HistoryActivity.kt

Didalam file **HistoryActivity**, import beberapa perintah, tambahkan deklarasi variable utk **tvDaftarRiwayat** untuk menampilkan data dari server, dan panggil **function ambilDataRiwayat()**

```
7      import androidx.core.view.WindowInsetsCompat
8
9      import android.widget.TextView
10     import android.widget.Toast
11     import retrofit2.Call
12     import retrofit2.Callback
13     import retrofit2.Response
14
15     <img alt="Run icon" data-bbox="75 535 105 555"/> </> class HistoryActivity : AppCompatActivity() {
```

```
14
15 <img alt="Run icon" data-bbox="510 285 540 305"/> </> class HistoryActivity : AppCompatActivity() {
16
17     private lateinit var tvDaftarRiwayat: TextView
18
19     <img alt="Run icon" data-bbox="525 425 545 445"/> override fun onCreate(savedInstanceState: Bundle?) {
```

```
22         setContentView(R.layout.activity_history)
23         > ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) {...}
28
29         tvDaftarRiwayat = findViewById(R.id.tvDaftarRiwayat)
30         <img alt="Lightbulb icon" data-bbox="210 795 225 815"/> ambilDataRiwayat()
31     }
```



HistoryActivity.kt

Dibawah **function onCreate()** tambahkan **function ambilDataRiwayat()** yang digunakan untuk mengambil data dari server

```
19  override fun onCreate(savedInstanceState: Bundle?) {...}
32
33  private fun ambilDataRiwayat() {
34      RetrofitClient.instance.ambilDataSensor().enqueue(object : Callback<List<SensorHistory>> {
35          override fun onResponse(call: Call<List<SensorHistory>>, response: Response<List<SensorHistory>>) {
36              if (response.isSuccessful) {
37                  val listData = response.body()
38                  if (!listData.isNullOrEmpty()) {
39                      val stringBuilder = StringBuilder()
40                      for (item in listData) {
41                          stringBuilder.append("Sumbu X: ${item.kemiringan_x}\n")
42                          stringBuilder.append("Status Setir: ${item.status_setir}\n")
43                          stringBuilder.append("Timestamp: ${item.timestamp}\n\n")
44                      }
45                      tvDaftarRiwayat.text = stringBuilder.toString()
46                  } else {
47                      tvDaftarRiwayat.text = "Tidak ada data riwayat."
48                  }
49              }
50          }
51      })
52  }
```



HistoryActivity.kt

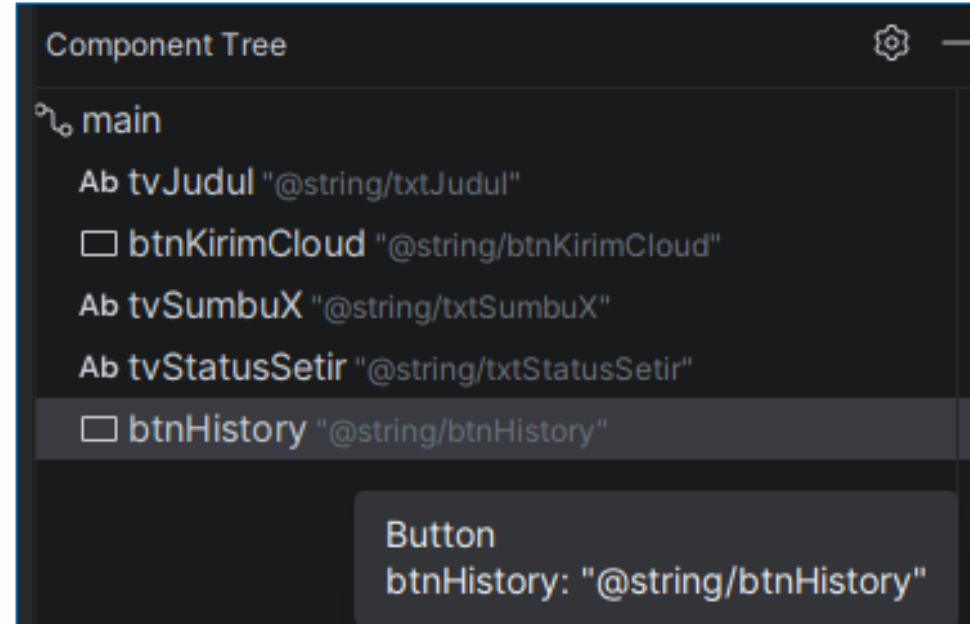
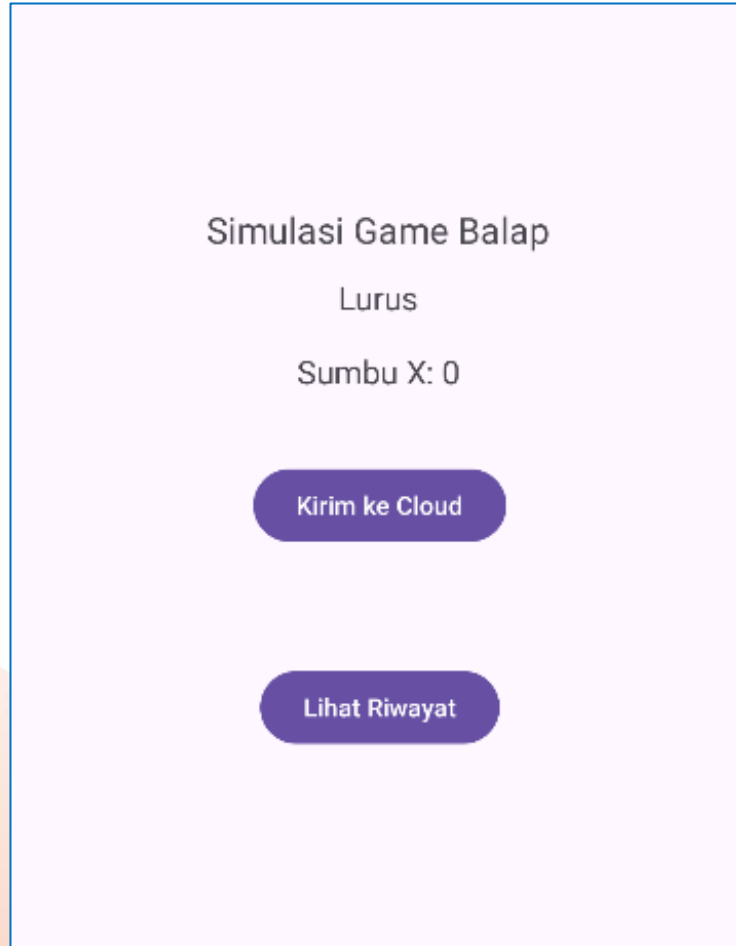
Berikut ini adalah lanjutan perintah didalam **function ambilDataRiwayat()**

```
33     private fun ambilDataRiwayat() {  
34         RetrofitClient.instance.ambilDataSensor().enqueue(object : Callback<List<SensorHistory>> {  
35             > override fun onResponse(call: Call<List<SensorHistory>>, response: Response<List<SensorHistory>>) {  
53  
54             > override fun onFailure(call: Call<List<SensorHistory>>, t: Throwable) {  
55                 tvDaftarRiwayat.text = "Koneksi error: ${t.message}"  
56                 Toast.makeText(this@HistoryActivity, "Koneksi error: ${t.message}", Toast.LENGTH_SHORT).show()  
57             }  
58         })  
59     }  
60 }
```



activity_accelerometer.xml

Tambahkan satu **Button** sebagai trigger pindah halaman ke **HistoryActivity**
Sesuaikan id Button tersebut





AccelerometerActivity.kt

Import perintah **Intent** untuk pindah halaman, tambahkan variable **Button** yg baru ditambahkan, dan tambahkan perintah untuk **Button** tersebut supaya pindah halaman ke **HistoryActivity**

```
20 import retrofit2.Response
21 
22 import android.content.Intent
23 
24 class AccelerometerActivity : AppCompatActivity
```

```
29 private lateinit var tvSumbuX: TextView
30 private lateinit var btnKirimCloud: Button
31 private lateinit var btnHistory: Button
32 
33 private var nilaiX: Float = 0f
```

```
52 > btnKirimCloud.setOnClickListener {...}
55 
56 btnHistory = findViewById(R.id.btnHistory)
57 btnHistory.setOnClickListener {
58     val intent = Intent(this, HistoryActivity::class.java)
59     startActivity(intent)
60 }
61 }
```



Simulasi Ambil Data

Run Aplikasi dan Klik tombol Lihat Riwayat untuk simulasi Ambil Data

